

SWENG3043

Operating Systems

Software Engineering Department

AASTU

Jan 2019

Chapter Eight

Input/output Management

Chapter Objectives

- To explore the structure of an operating system's I/O subsystem.
- To discuss the principles and complexities of I/O hardware.
- To explain the performance aspects of I/O hardware and software.

Topics Included

- Introduction
- Principles of I/O hardware
- Principles of I/O software

Introduction

- One of the main functions of an operating system is to control all the computer's I/O (Input/Output) devices.
- I/O is important for communication between the user and computer.
- I/O devices vary so widely in their function and speed, varied methods are needed to control them.
- The following are functions of the operating system as I/O manager:
 - Control all the computer I/O devices.
 - Issue commands to I/O devices, catch interrupts and handle errors.
 - Provide an interface between the device and the rest of the system.

Principles of I/O Hardware

- Different people look I/O hardware in different ways.
 - Electrical engineers look at it in terms of chips, wires, power supplies, motors, and all the other physical components that make up the hardware.
 - Programmers look at the interface presented to the software
 - The commands the hardware accepts, the functions it carries out, and the errors that can be reported back.
 - our interest will be restricted to how the hardware is programmed, not how it works inside.
- There are two types of I/O devices: block and character
 - **Block devices** (such as disks) - is one that stores information in fixed-size blocks, each one with its own address.

I/O Devices(cont'd)

- Common block size range from 512 bytes to 32,768 bytes.
- The essential property of a block device is that it is possible to read or write each block **independently** of all the other ones.
- Disks are the most common block devices.
- A disk is a block addressable device because no matter where the arm currently is, it is always possible to seek to another cylinder and then wait for the required block to rotate under the head.
- **Character devices** (such as printers, mouse and NIC)
 - A character device delivers or accepts stream of character, without regard to any block structure.
 - It is not addressable and does not have any seek operation.
- I/O devices vary so widely in their function and speed
 - Varied methods are needed to control them

I/O Devices(cont'd)

Device	Data rate
Keyboard	10 bytes/sec
Mouse	100 bytes/sec
56K modem	7 KB/sec
Telephone channel	8 KB/sec
Dual ISDN lines	16 KB/sec
Laser printer	100 KB/sec
Scanner	400 KB/sec
Classic Ethernet	1.25 MB/sec
USB (Universal Serial Bus)	1.5 MB/sec
Digital camcorder	4 MB/sec
IDE disk	5 MB/sec
40x CD-ROM	6 MB/sec
Fast Ethernet	12.5 MB/sec
ISA bus	16.7 MB/sec
EIDE (ATA-2) disk	16.7 MB/sec
FireWire (IEEE 1394)	50 MB/sec
XGA Monitor	60 MB/sec
SONET OC-12 network	78 MB/sec
SCSI Ultra 2 disk	80 MB/sec
Gigabit Ethernet	125 MB/sec
Ultrium tape	320 MB/sec
PCI bus	528 MB/sec
Sun Gigaplane XB backplane	20 GB/sec

Some typical device, network, and data rates

Device Controllers

- I/O devices have components:

- mechanical component

- The mechanical component is the device itself

- electronic component

- The electronic component is the **device controller**/adapter

- Controller's tasks

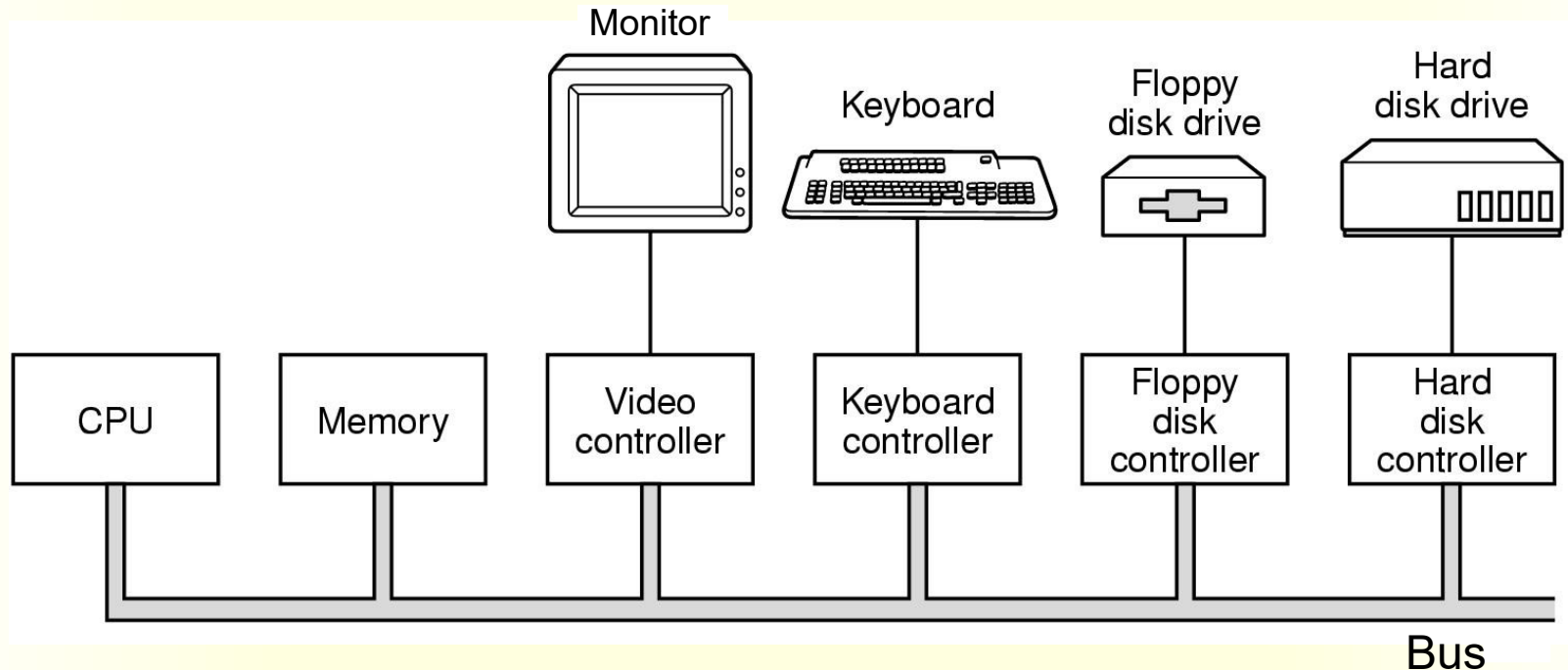
- convert serial bit stream to block of bytes

- perform error correction as necessary

- make available to main memory

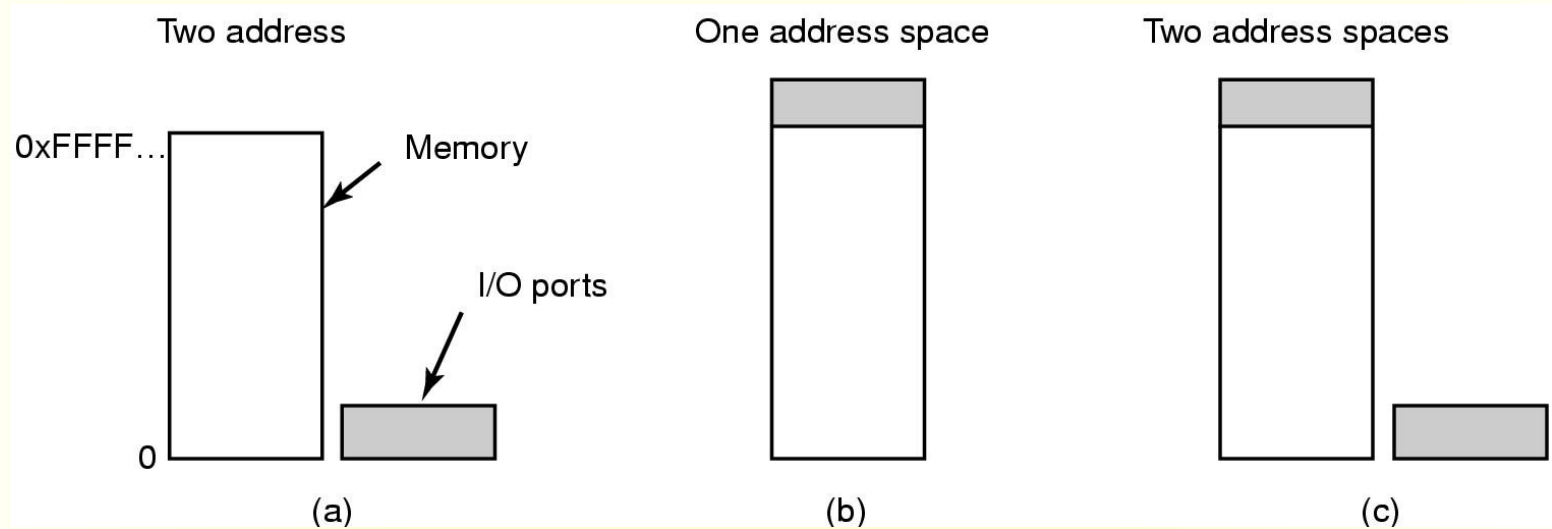
- Each controller has a few registers that are used for communicating with the CPU.

Device Controllers(cont'd)



A model for connecting the CPU, memory, controllers, and I/O devices

Memory-Mapped I/O



- Controller has one or more registers for data and control signals. Processor communicates with control register in one of those methods:
 - Separate I/O and memory space
 - Memory-mapped I/O- device-control registers are mapped into the address space of the processor
 - Hybrid

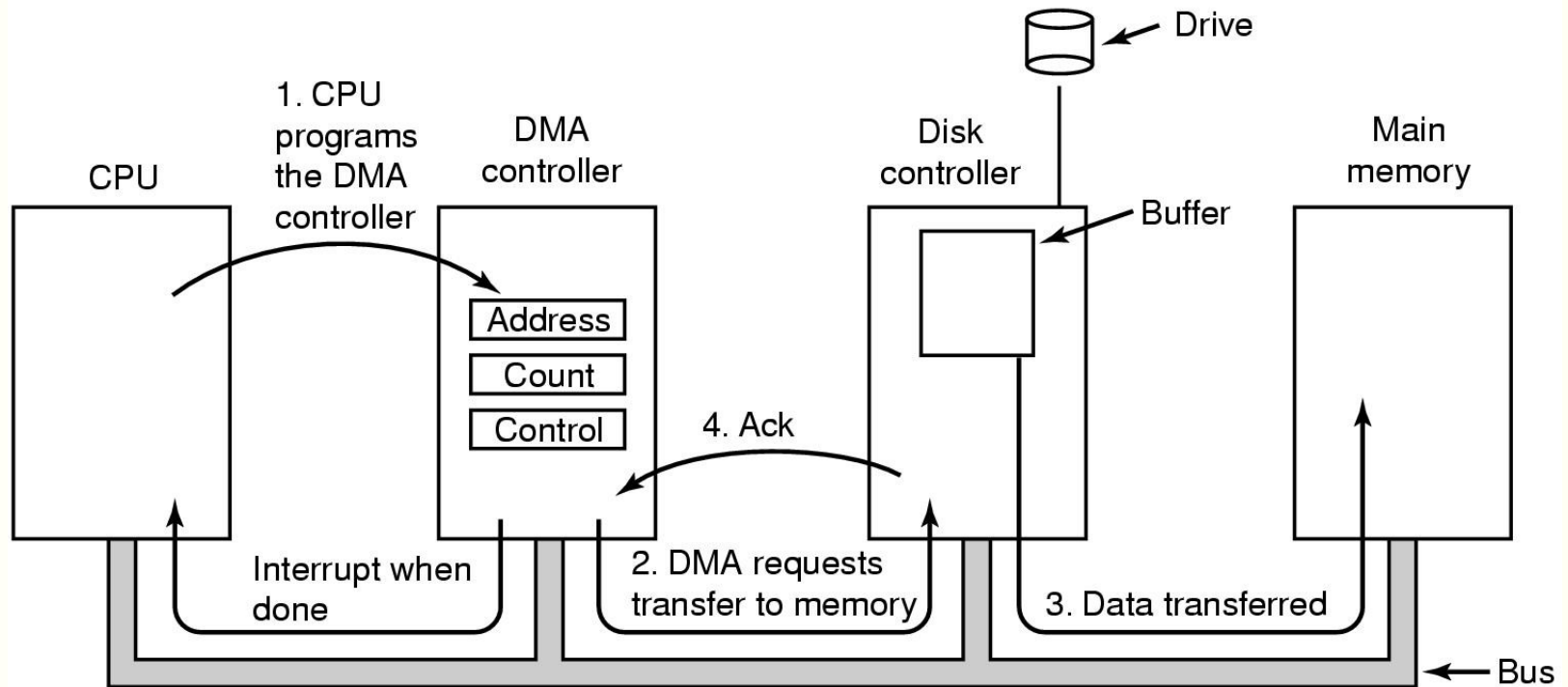
Direct Memory Access(DMA)

- A technique for moving a data directly between main memory and I/O devices without the CPU's intervention.
- The CPU can request data from an I/O controller one byte at a time but doing so for a device like a disk that produces a large block of data **wastes the CPU's time.**
- In the absence of DMA reading from a disk is done with the following steps:
 - The controller reads the block (one or more sectors) from the drive serially, bit by bit, until the entire block is in the controller's internal buffer
 - The controller computes the checksum to verify that no read errors have occurred

DMA(cont'd)

- The controller causes an interrupt
- The OS reads the disk block from the controller's buffer a byte or a word at a time and stores it on memory
- **Problem:** Since the OS controls the loop of reading it wastes the CPU time.
- When DMA is used,
 - CPU programs the DMA controller so it knows what to transfer where.
 - Tells disk controller to read data from the disk into its internal buffer and verify the checksum.

DMA(cont'd)



Operation of a DMA transfer

Principles of I/O Software

■ Goals and issues of I/O software:

■ Device Independence:

- It should be possible to write programs that can read files on hard disk or on a CD-ROM, without having to modify the program for each different device types.
- It is up to the operating system to take care of the problems caused by the fact that these devices are really different.

■ Uniform Naming: the name of a file or a device should simply be a string or an integer and not dependant on the device in any way.

- E.g. a file called IAmStoredOnAHardDisk might well be stored on a CD

Principles of I/O Software(cont'd)

■ Error handling:

- In general, errors should be handled as close to the hardware as possible (at the device controller level).
- Only if the lower layers are not be able to deal with the problem should the upper layers be told about it.

■ Transfer:

- There are two types of transfer modes – **Synchronous (Blocking)** and **Asynchronous (interrupt –driven)**.
- In the case of synchronous transfer the program requesting I/O transfer will be suspended until the transfer is completed.
- In the case of Asynchronous transfer the CPU starts the transfer and goes off to do something until the interrupt that shows the completion of the transfer arrives.

Principles of I/O Software(cont'd)

■ Buffering

- data coming off a device cannot be stored in final destination
- For example, when a packet comes in off the network, the operating system does not know where to put it until it has stored the packet somewhere and examined it.

■ **Device types:** There are two device types

- **Sharable** (such as disk) - used by many users at the same time. No problems are caused by multiple users having open files on the same disk at the same time.
- **Dedicated** (tape drives) have to be dedicated to a single user until that user is finished. Having two or more users writing blocks intermixed at random to the same tape will definitely not work.

Layers of I/O Software

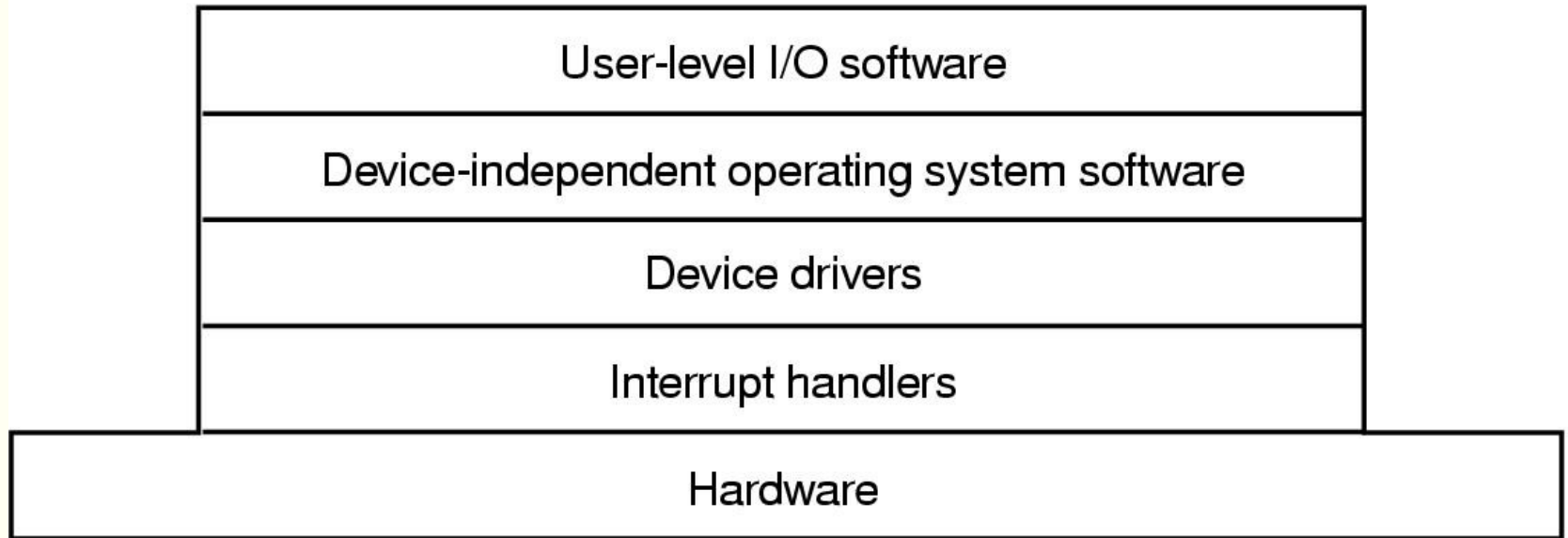
- The following are the I/O software layers

- Interrupt handler (bottom)
- Device driver
- Device independent OS software
- User-level software (top)

- **Interrupt Handler**

- Interrupts are unpleasant facts of life and cannot be avoided. They should be hidden away, so that as little of the operating system as possible knows about them.
- The best way to hide them is to have every process starting an I/O operation block until the I/O has completed and the interrupt occurs.
- When the interrupt happens, the interrupt procedure does whatever it has to in order to unblock the process that started it.

Layers of I/O Software(cont'd)



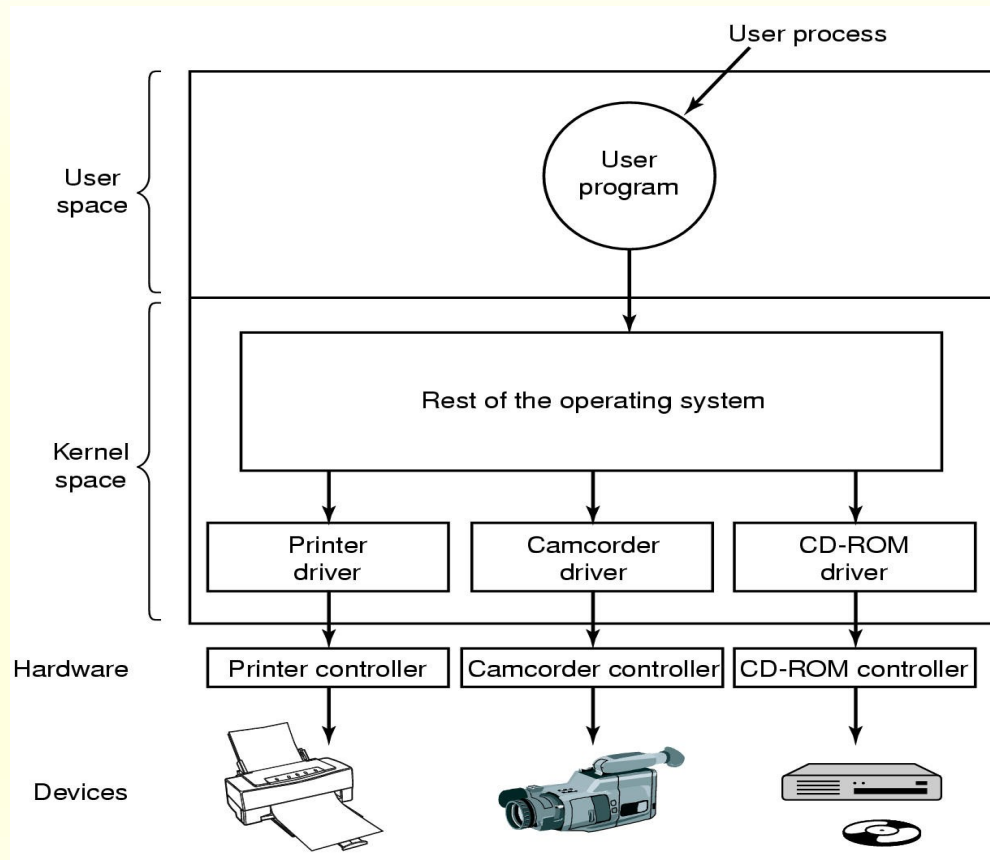
Layers of the I/O Software System

Layers of I/O Software(cont'd)

■ Device Driver

- Is a device – specific code for controlling I/O devices.
 - each device controller has registers used to give it commands or to read out its status. The number of registers and the nature of the commands vary radically from device to device.
 - For example, a mouse driver has to accept information from the mouse telling how far it has moved and which buttons are currently depressed.
 - In contrast, a disk driver has to know about sectors, tracks, cylinders, heads, arm motion, motor drives, head settling times, and all the other mechanics of making the disk work properly. **Obviously, these drivers will be very different.**
- Each device driver handles one device type, or at most, one class of closely related devices.
- In general terms, the job of a device driver is to accept requests from the device-independent software above it and sees to it that the request is executed.
- Thus, the disk driver is the only part of the OS that knows how many registers that disk controller has and what they are used for.

Layers of I/O Software(cont'd)



■ Logical position of device drivers is shown here

Layers of I/O Software(cont'd)

■ Steps in carrying out I/O requests:

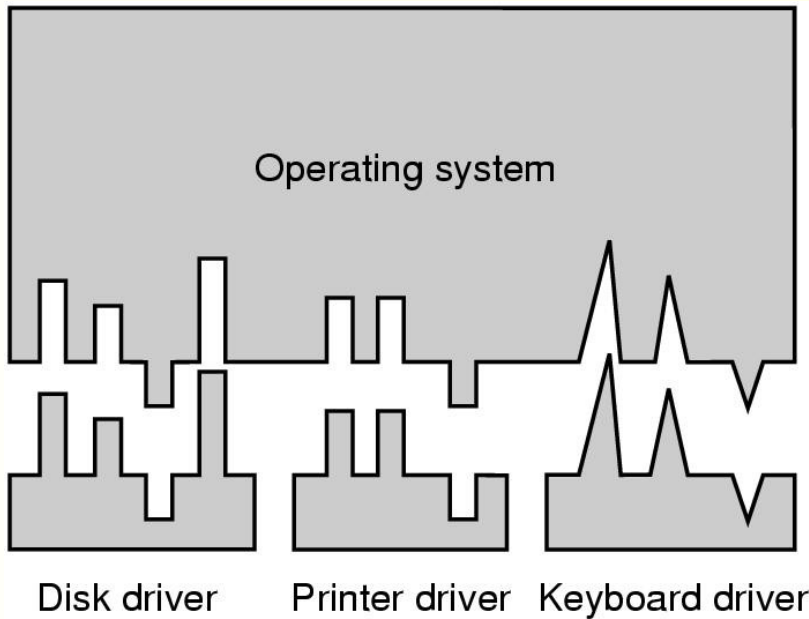
- Translate it from abstract to concrete terms
 - E.g. For disk driver: convert a linear block number into the head, track, sector, and cylinder numbers for disk's geometry.
- Write into the controller's device registers
- The device driver blocks itself until interrupt comes
- The blocked driver will be awakened by the interrupt
- Device driver checks for errors
- If everything is all right, the driver may have data to pass to the device independent software
- If nothing is queued, the driver blocks waiting for the next request

Layers of I/O Software(cont'd)

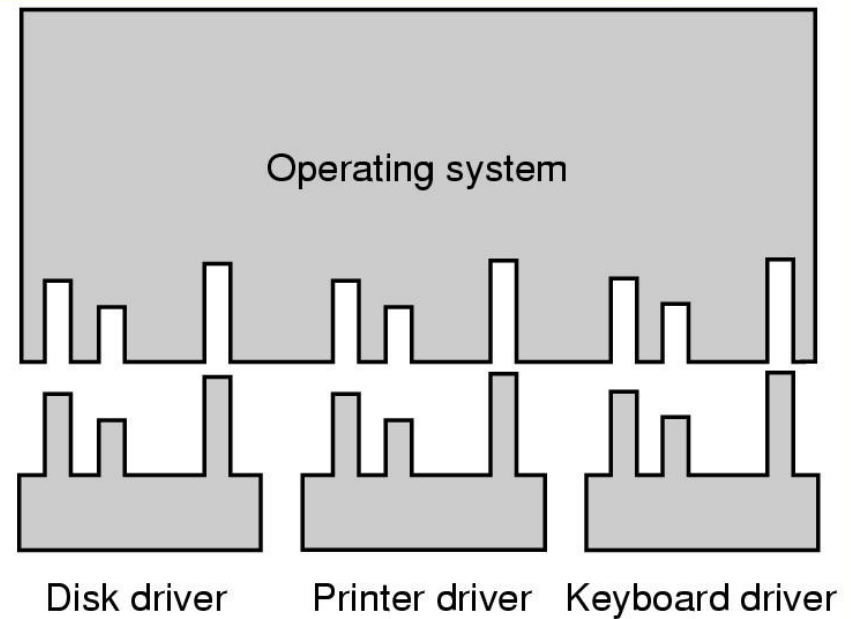
■ Device Independent I/O Software

- It is large fraction of I/O software
- Perform the I/O functions that are common to all devices and to provide a uniform interface to the user-level software.
- Functions of the device-independent I/O software
 - **Uniform interfacing for device drivers**
 - A major issue in an operating system is how to make all I/O devices and drivers look more-or-less the same.
 - If disks, printers, monitors, keyboards, etc., are all interfaced in different ways, **every time a new peripheral device comes along, the operating system must be modified for the new device.**
 - With a standard interface it is much easier to plug in a new driver. It also means that driver writers know what is expected of them (e.g., what functions they must provide and what kernel functions they may call).
 - **Device Naming** – responsible for mapping symbolic devices names onto the proper driver

Layers of I/O Software(cont'd)



(a)



(b)

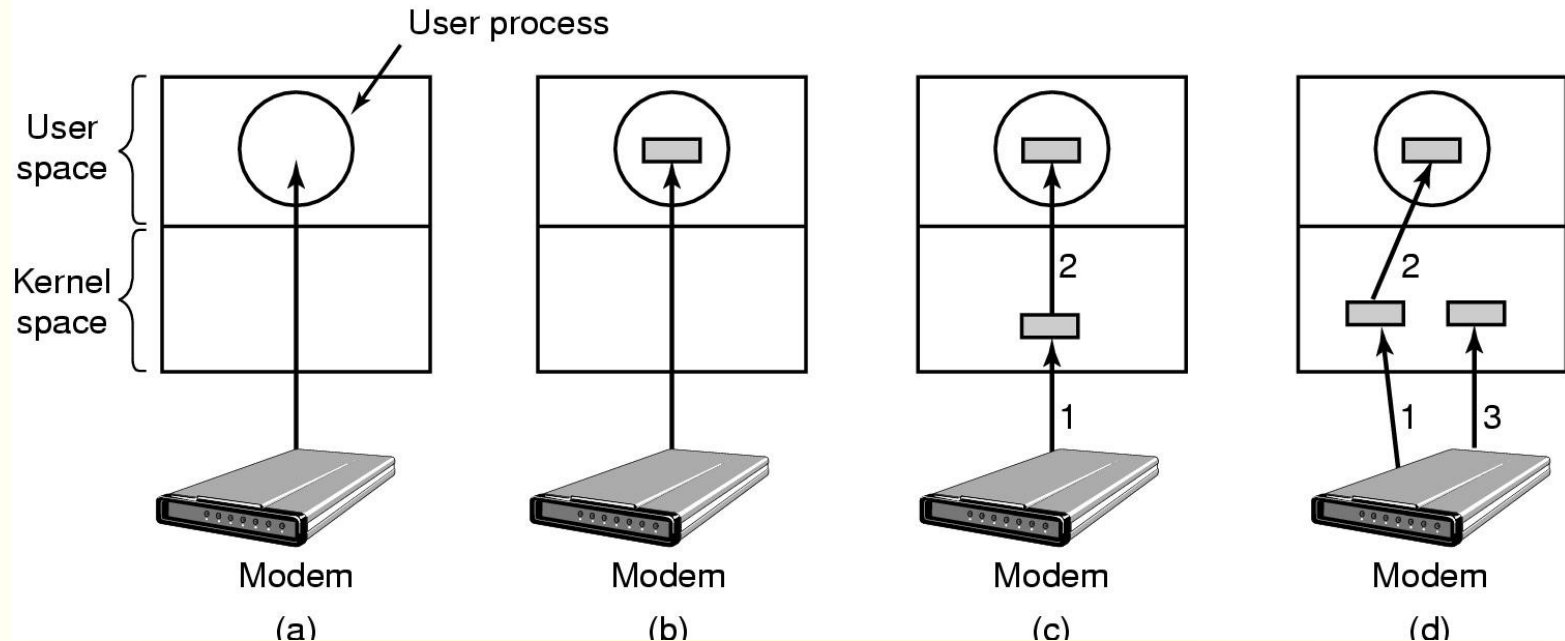
(a) Without a standard driver interface

(b) With a standard driver interface

Layers of I/O Software(cont'd)

- **Providing device-independent block size** – provide uniform block size to higher layers hiding differences in block sizes.
 - Different disks may have different sector size.
 - Some devices deliver their data one byte at a time(e.g., modems), while others deliver theirs in large units(e.g., network interfaces).
- **Buffering:** if a user process write half a block, the OS will normally keep the data in buffer until the rest of the data are written.
- **Device protection** – prevent users from accessing devices that they are not entitled to access
- **Allocating and releasing dedicated devices:** It is up to the OS to examine requests for devices usage and accept or reject them.
- **Error reporting:** Errors handling, by and large, is done by drivers. Most errors are device dependent. After the driver tries to read the block a certain number of times, it gives up and informs the device-independent software. It then reports to the caller.

Layers of I/O Software(cont'd)



(a) Unbuffered input

(b) Buffering in user space

(c) Buffering in the kernel followed by copying to user space

(d) Double buffering in the kernel

Layers of I/O Software(cont'd)

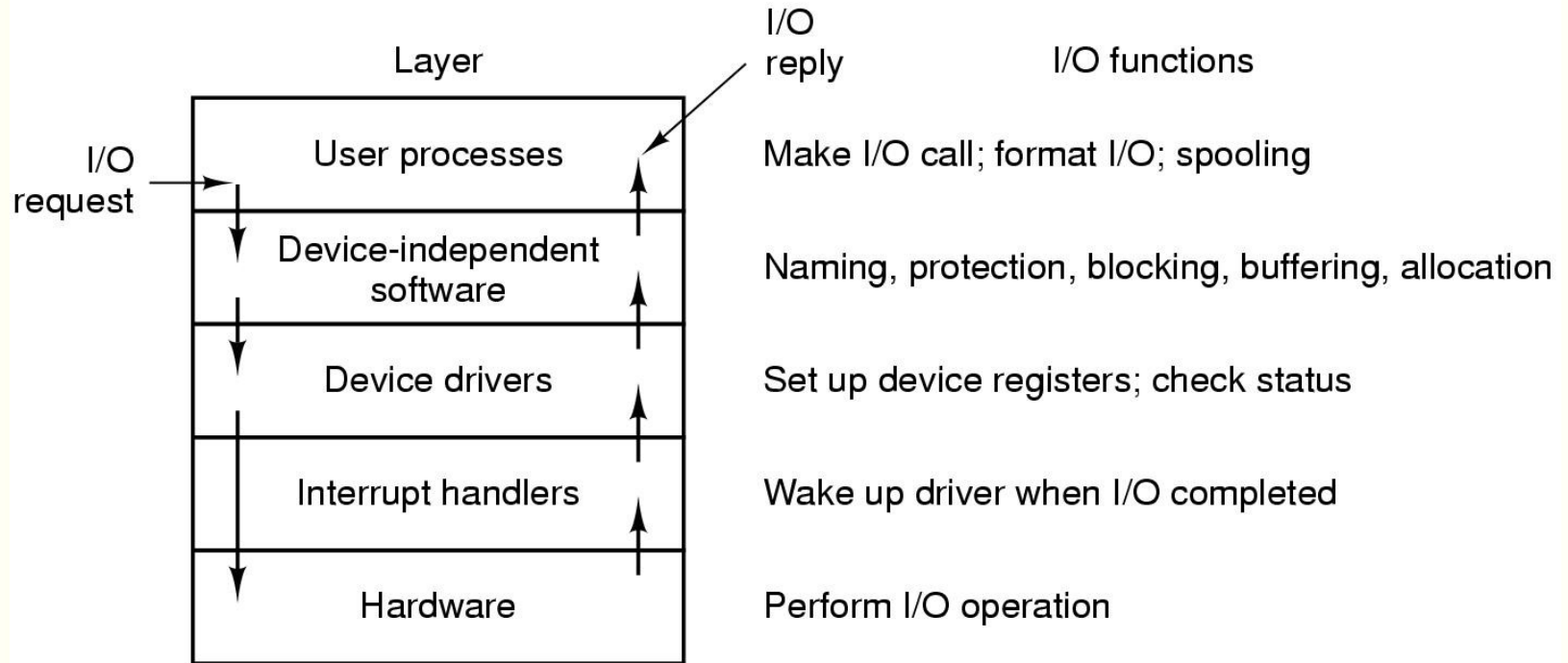
■ User Space I/O Software

- A small portion of the I/O software is outside the OS
- System calls, including the I/O system calls, are normally made by library procedures.
- E.g. when a C program contains the call `count = write (fd, buffer, nbytes);` the library procedure `write` will be linked with the program and contained in the binary program present in memory at run time.
- Formatting of input and output is done by library procedures
 - E.g `printf("Value of i is: =%d\n", i);`
 - `scanf("%d %d", &fnumber, &snumber);`

Layers of I/O Software(cont'd)

- The following is an example how the I/O system works during reading by an application
 - **Step 1:** the user program produce a system call for reading from a file
 - **Step 2:** the device-independent software looks in the block cache; if the needed block is not there it calls the device driver.
 - **Step 3:** The device driver issue the request to the hardware and the process will be blocked until the disk operation has been completed
 - **Step 4:** When the disk is finished, the hardware generates an interrupt
 - **Step 5:** The interrupt handler is run to discover what happened, i.e., which device wants attention right now. It then extracts the status form the device and wakes up the sleeping process to finish off the I/O request and let the user process continue

Layers of I/O Software(cont'd)



Layers of the I/O system and the main functions of each layer